Divide and Conquer

Introduction

- Divide and Conquer algorithms consist of two parts:
  - ***Divide:*** Smaller problems are solved recursively (except, of course, the base cases).
  - ***Conquer:*** The solution to the original problem is then formed from the solutions to the subproblems.
- Examples
  - Binary Search
  - Quick Sort
  - Merge Sort
  - Long Integer Multiplication
  - Maximum Sum Subarray
  - Strassen's Algorithm for Matrix Multiplication etc.

Integer Multiplication

```
  1011
 x1111
  1011
 10110
101100
+1011000
10100101
```

- The standard integer multiplication routine of 2 n-digit numbers
  - Involves n multiplications of an n-digit number by a single digit
  - Plus the addition of n numbers, which have at most 2 n digits

		quantity	time
1)	Multiplication n-digit by 1-digit	n	O(n)
2)	Additions 2n-digit by n-digit max	n	O(n)

Total time = $n*O(n) + n*O(n) = 2n*O(n) = O(n^2)$

Integer Multiplication

- Let's consider a Divide and Conquer Solution
  - Imagine multiplying an n-bit number by another n-bit number.
    - We can split up each of these numbers into 2 halves.
    - Let the 1st number be I, and the 2nd number J
    - Let the "left half" of the 1st number by I_h and the "right half" be I_l.
  - So in this example: I is 1011 and J is 1111
    - I becomes $10*2^2 + 11$ where $I_h = 10*2^2$ and $I_l = 11$.
    - and $J_h = 11*2^2$ and $J_l = 11$

```
  1011
 x1111
  1011
 10110
101100
+1011000
10100101
```

Integer Multiplication

- So for multiplying any n-bit integers I and J
  - We can split up I into $(I_h * 2^{n/2}) + I_l$
  - And J into $(J_h * 2^{n/2}) + J_l$
- Then we get
  - $I \times J = [(I_h \times 2^{n/2}) + I_l] \times [(J_h \times 2^{n/2}) + J_l]$
  - $I \times J = I_h \times J_h \times 2^n + (I_l \times J_h + I_h \times J_l) \times 2^{n/2} + I_l \times J_l$

- So what have we done?
  - We've broken down the problem of multiplying 2 n-bit numbers into
    - 4 multiplications of n/2-bit numbers plus 3 additions.
  - $T(n) = 4T(n/2) + \theta(n)$
  - Solving this using the master theorem gives us…
    - $T(n) = \theta(n^2)$

Integer Multiplication (Karatsuba Algorithm)

- So we haven't really improved that much,
  - Since we went from a $O(n^2)$ solution to a $O(n^2)$ solution
- Can we optimize this in any way?
  - We can re-work this formula using some clever choices…
  - Some clever choices of:

$$P_1 = (I_h + I_l) \times (J_h + J_l) = I_h x J_h + I_h x J_l + I_l x J_h + I_l x J_l$$

$$P_2 = I_h \times J_h, \text{ and}$$

$$P_3 = I_l \times J_l$$

  - Now, note that

$$P_1 - P_2 - P_3 = I_h x J_h + I_h x J_l + I_l x J_h + I_l x J_l - I_h x J_h - I_l x J_l$$

$$= I_h x J_l + I_l x J_h$$

  - Then we can substitute these in our original equation:

$$I \times J = P_2 \times 2^n + [P_1 - P_2 - P_3] \times 2^{n/2} + P_3.$$

Integer Multiplication

$$(I_h - I_l) \cdot (J_l - J_h) = I_h J_l - I_l J_l - I_h J_h + I_l J_h.$$

$$I \cdot J = I_h J_h 2^n + [(I_h - I_l) \cdot (J_l - J_h) + I_h J_h + I_l J_l] 2^{n/2} + I_l J_l.$$

$$I \times J = P_2 \times 2^n + [P_1 - P_2 - P_3] \times 2^{n/2} + P_3.$$

- Have we reduced the work?
  - Calculating P2 and P3 – take n/2-bit multiplications.
  - P1 takes two n/2-bit additions and then one n/2-bit multiplication.
  - Then, 2 subtractions and another 2 additions, which take O(n) time.

- This gives us : $T(n) = 3T(n/2) + \theta(n)$
  - Solving gives us $T(n) = \theta(n^{(\log_2 3)})$, which is approximately $T(n) = \theta(n^{1.585})$, a solid improvement.

Integer Multiplication

- Although this seems it would be slower initially because of some extra pre-computing before doing the multiplications, ***for very large integers***, this will save time.

- Q: Why won't this save time for small multiplications?
  - A: The hidden constant in the θ(n) in the second recurrence is much larger. It consists of 6 additions/subtractions whereas the θ(n) in the first recurrence consists of 3 additions/subtractions.

Integer Multiplication - Example

$I \quad J$

1101×1111

$I = I_h I_l = 11 * 2^2 + 01$

$J = J_h J_l = 11 \times 2^2 + 11$

$$I \cdot J = (I_h J_h)\, 2^n + \left[(I_h + I_l) \cdot (J_h + J_l) - I_h J_h - I_l J_l\right] 2^{n/2} + I_l J_l \quad \text{---(1)}$$

Integer Multiplication - Example

$$1101 * 1111 = (11 \cdot 11)\,2^4 + [\,(11+01)\cdot(11+11) - 11\cdot11 - 01\cdot01\,]\,2^2 + 01\cdot01 \quad \text{—(2)}$$

(0100 and 0110 above the middle terms)

$$11\cdot11 = (1\cdot1)\,2^2 + [\,(1+1)\cdot(1+1) - 1\cdot1 - 1\cdot1\,]\,2^1 + 1\cdot1$$

(10 · 10 under $(1+1)\cdot(1+1)$)

$$= 100 + [\,100 - 1 - 1\,]\,2^1 + 1$$

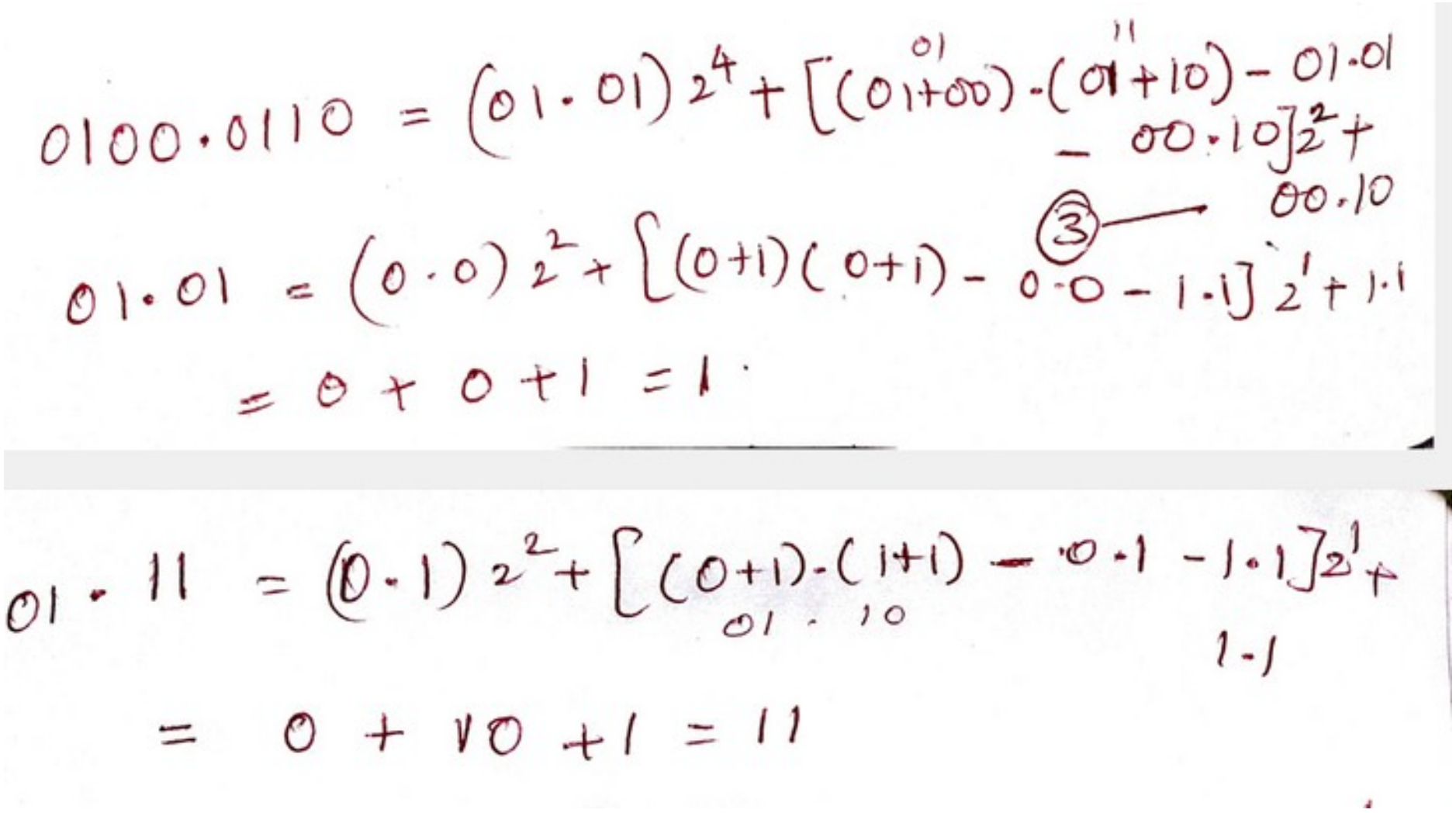
$$= 100 + 100 + 1 = 1001$$

$$10\cdot10 = (1\cdot1)\,2^2 + [\,(1+0)\cdot(1+0) - 1\cdot1 - 0\cdot0\,]\,2^1 + 0\cdot0$$

(1 · 1 under $(1+0)\cdot(1+0)$)

$$= 100 + 0 + 0 = 100$$

Integer Multiplication - Example



Integer Multiplication - Example

$01.10 = (0.1)\,2^2 + [(0+1)\cdot(1+0) - 0\cdot1 - 1\cdot0]\,2^1 + 1\cdot0$

$= 0 + 10 + 0 = 10$

$00.10 = (0.1)\,2^2 + [(0+0)\cdot(1+0) - 0\cdot1 - 0\cdot0]\,2^1 + 0\cdot0$

$= 0 + 0 + 0 = 0$

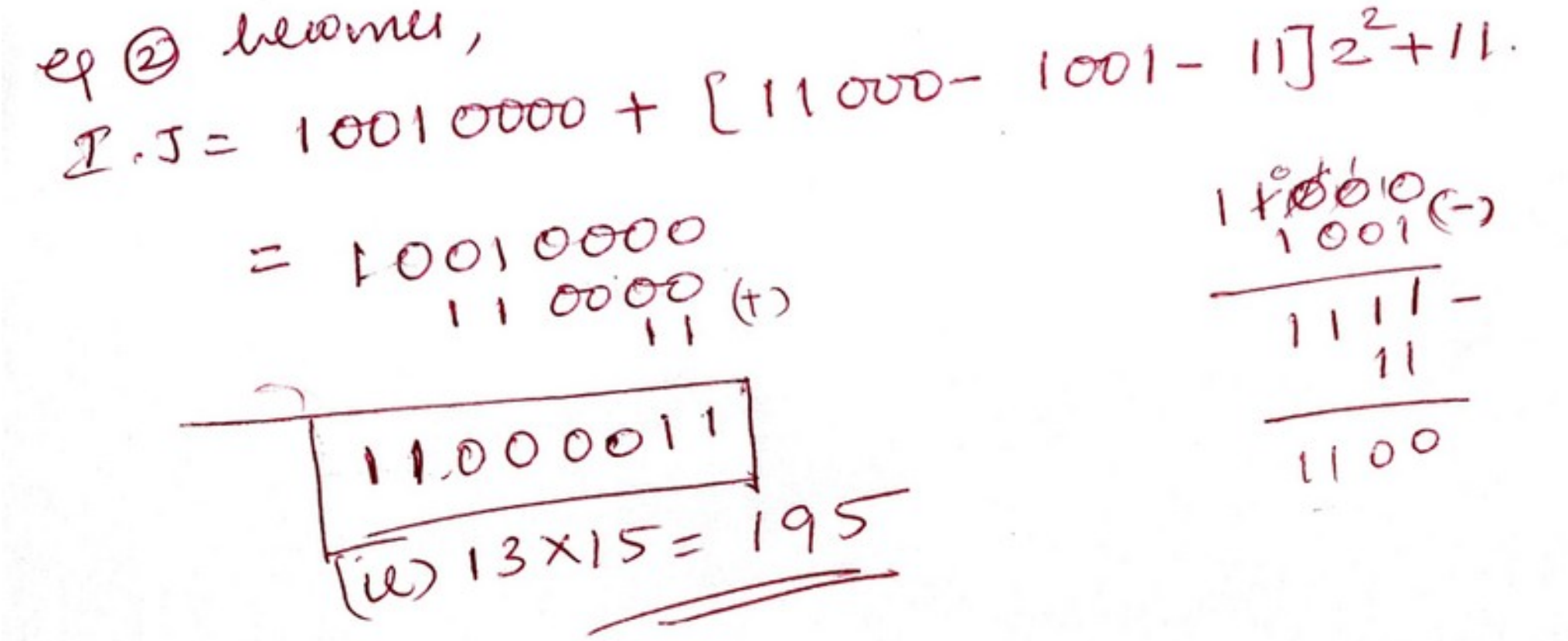
eq (3) becomes;

$0100 \cdot 0110 = 10000 + (11 - 1000 - 0)\,2^2 + 0$

$= 10000 + 1000 + 0$

$= 11000$

Integer Multiplication - Example



Pseudocode

Figure 2.1 A divide-and-conquer algorithm for integer multiplication.

```
function multiply(x, y)
Input:  Positive integers x and y, in binary
Output:  Their product

n = max(size of x, size of y)
if n = 1:  return xy

x_L, x_R = leftmost ⌈n/2⌉, rightmost ⌊n/2⌋ bits of x
y_L, y_R = leftmost ⌈n/2⌉, rightmost ⌊n/2⌋ bits of y

P_1 = multiply(x_L, y_L)
P_2 = multiply(x_R, y_R)
P_3 = multiply(x_L + x_R, y_L + y_R)
return  P_1 × 2^n + (P_3 − P_1 − P_2) × 2^(n/2) + P_2
```

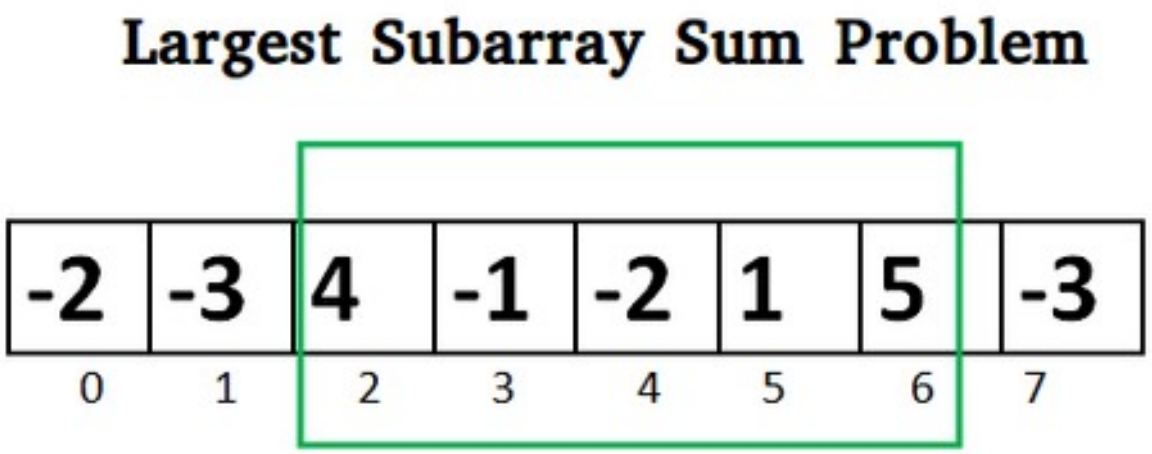
Long Integer Multiplication

- Solve
  - 1101 x 1011
  - 1234 x 2345

Maximum Sum Subarray

- The **maximum subarray problem** is the task of finding the **largest** possible **sum** of a **contiguous subarray**, within a given one-dimensional array A[1…n] of numbers.
- The time **complexity** of the Naive method is O(n^2).
- Using Divide and Conquer approach, we can find the **maximum subarray sum** in O(nLogn) time.

Maximum Sum Subarray



4 + (-1) + (-2) + 1 + 5 = 7

Maximum Contiguous Array Sum is 7

Solution #1 - Brute Force

- **The naive approach is to generate all the possible subarray and print that subarray which has maximum sum.**

```
int MSS (int a[], int n)
{  int max=0
   for(i=0 to n-1 do)
      for(j=i to n-1 do)
         int s=0
         for (k=i to j do)
            s=s+a[k]
         if(s>max)
           max= s
   return max  }
```

$O(n^3)$

```
int MSS (int a[], int n)
 {
   int max=0
   for(i=0 to n-1 do)
        int s=0
        for (k=i to n do)
            s=s+a[k]
            if(s>max)
                 max= s
   return max
}
```

$O(n^2)$

Solution #2 – Divide and Conquer

```
/ Returns sum of maxium sum subarray in aa[l..h]
int maxSubArraySum(int arr[], int l, int h)
{
    // Base Case: Only one element
    if (l == h)
        return arr[l];


    // Find middle point
    int m = (l + h) / 2;


    /* Return maximum of following three possible
cases
            a) Maximum subarray sum in left half
            b) Maximum subarray sum in right half
            c) Maximum subarray sum such that the
subarray
       crosses the midpoint */
    return max(maxSubArraySum(arr, l, m),
               maxSubArraySum(arr, m + 1, h),
               maxCrossingSum(arr, l, m, h));
}
```

Maximum Sum Subarray

```
// Find the maximum possible sum in arr[] auch that arr[m] is part of it
int maxCrossingSum(int arr[], int l, int m, int h)
{
    // Include elements on left of mid.
    int sum = 0;
    int left_sum = INT_MIN;
    for (int i = m; i >= l; i--) {
        sum = sum + arr[i];
        if (sum > left_sum)
            left_sum = sum;
    }
    // Include elements on right of mid
    sum = 0;
    int right_sum = INT_MIN;
    for (int i = m + 1; i <= h; i++) {
        sum = sum + arr[i];
        if (sum > right_sum)
            right_sum = sum;
    }
    // Return sum of elements on left and right of mid returning only left_sum +

    //right_sum will fail for [-2, 1]
    return max(left_sum + right_sum, left_sum, right_sum);
}
```

Maximum Sum Subarray - Example

- 2, -4, 1, 9, -6, 7, -3 Soln: 11

Maximum Sum Subarray - Example

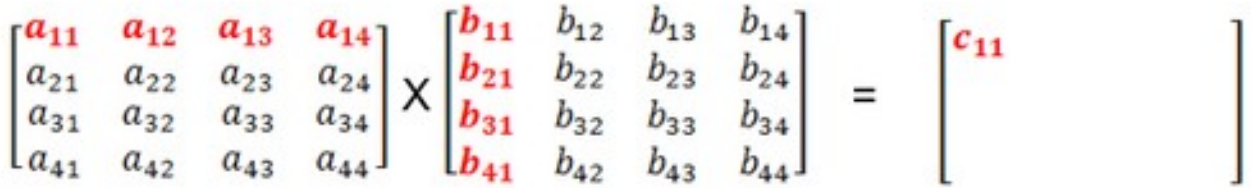
- 3, -5, 1, -2, 6, 4, -3 Soln: 10

Maximum Sum Subarray - Example

- 2, -5, 6, -2, -3, 1, 5, -6 Soln: 7

Strassen's Algorithm

- A fundamental numerical operation is the multiplication of 2 matrices.
  - The standard method of matrix multiplication of two n x n matrices takes T(n) = O(n3).



The following algorithm multiples n x n matrices A and B:

```
// Initialize C.
for i = 1 to n
  for j = 1 to n
    for k = 1 to n
        C [i, j] += A[i, k] * B[k, j];
```

Strassen's Algorithm

- We can use a Divide and Conquer solution to solve matrix multiplication by separating a matrix into 4 quadrants:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \\ b_{31} & b_{32} & b_{33} & b_{34} \\ b_{41} & b_{42} & b_{43} & b_{44} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & c_{13} & c_{14} \\ c_{21} & c_{22} & c_{23} & c_{24} \\ c_{31} & c_{32} & c_{33} & c_{34} \\ c_{41} & c_{42} & c_{43} & c_{44} \end{bmatrix}$$

- Then we know have:

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \quad B = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} \quad C = \begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix}$$

if $C = AB$ **, then we have the following:**

$$c_{11} = a_{11}b_{11} + a_{12}b_{21}$$
$$c_{12} = a_{11}b_{12} + a_{12}b_{22}$$
$$c_{21} = a_{21}b_{11} + a_{22}b_{21}$$
$$c_{22} = a_{21}b_{12} + a_{22}b_{22}$$

8 n/2 * n/2 matrix multiples + 4 n/2 * n/2 matrix additions

$T(n) = 8T(n/2) + O(n^2)$

If we solve using the master theorem we still have $O(n^3)$

Strassen's Algorithm

- Strassen showed how two matrices can be multiplied using only 7 multiplications and 18 additions:
  - Consider calculating the following 7 products:
    - $q_1 = (a_{11} + a_{22}) * (b_{11} + b_{22})$
    - $q_2 = (a_{21} + a_{22}) * b_{11}$
    - $q_3 = a_{11} * (b_{12} - b_{22})$
    - $q_4 = a_{22} * (b_{21} - b_{11})$
    - $q_5 = (a_{11} + a_{12}) * b_{22}$
    - $q_6 = (a_{21} - a_{11}) * (b_{11} + b_{12})$
    - $q_7 = (a_{12} - a_{22}) * (b_{21} + b_{22})$
  - It turns out that
    - $c_{11} = q_1 + q_4 - q_5 + q_7$
    - $c_{12} = q_3 + q_5$
    - $c_{21} = q_2 + q_4$
    - $c_{22} = q_1 + q_3 - q_2 + q_6$

Strassen's Algorithm

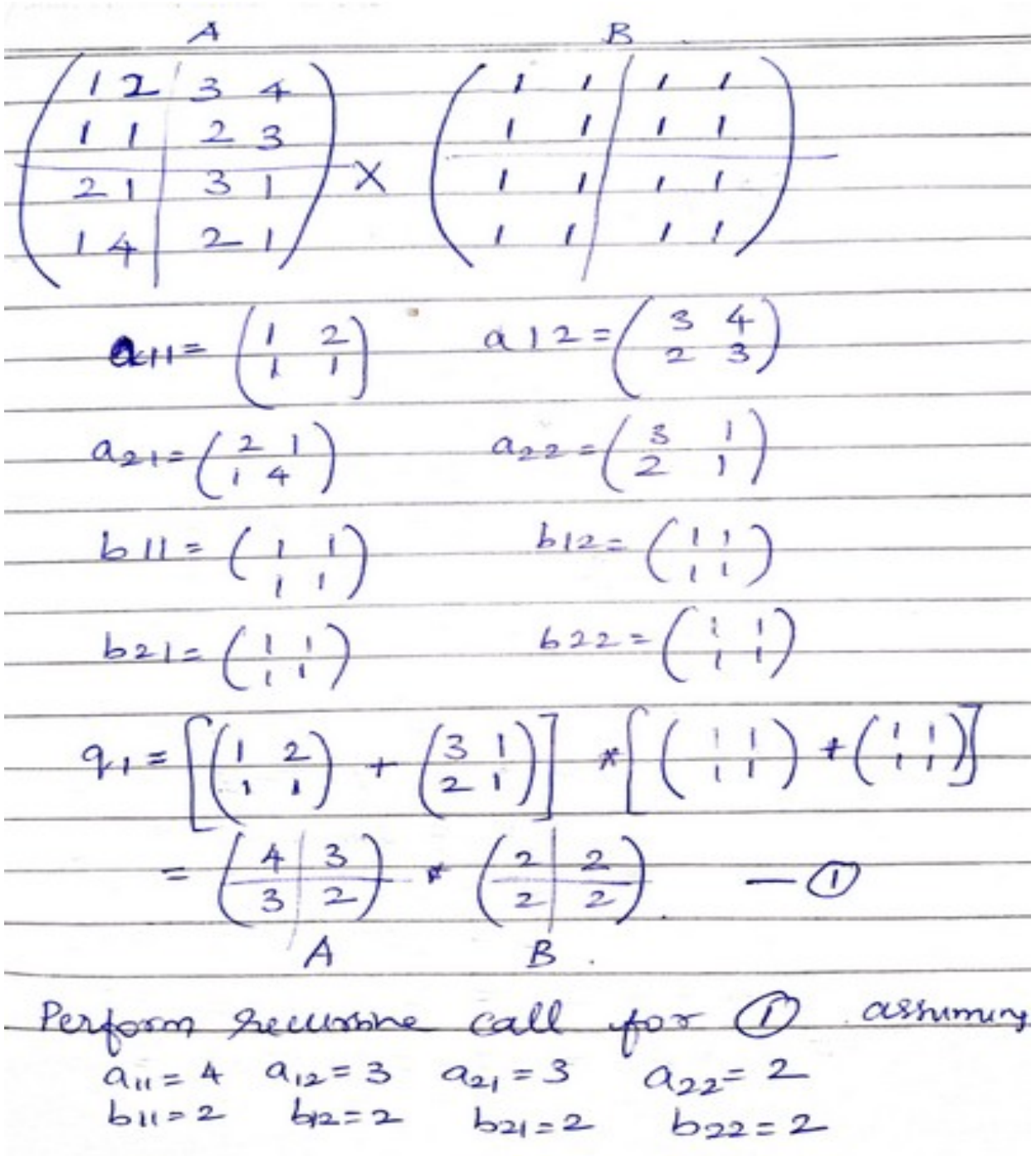
- Let's verify one of these:

  Given: $A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}$ $B = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}$ $C = \begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix}$

  if $C = AB$, we know: $c_{21} = a_{21}b_{11} + a_{22}b_{21}$

- Strassen's Algorithm states:
  - $\mathbf{c_{21} = q_2 + q_4}$,
    where $\mathbf{q_4 = a_{22} * (b_{21} - b_{11})}$ and $\mathbf{q_2 = (a_{21} + a_{22}) * b_{11}}$

Example


A
B
$a_{11} = \begin{pmatrix} 1 & 2 \\ 1 & 1 \end{pmatrix}$ $a_{12} = \begin{pmatrix} 3 & 4 \\ 2 & 3 \end{pmatrix}$
$a_{21} = \begin{pmatrix} 2 & 1 \\ 1 & 4 \end{pmatrix}$ $a_{22} = \begin{pmatrix} 3 & 1 \\ 2 & 1 \end{pmatrix}$
$b_{11} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ $b_{12} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$
$b_{21} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ $b_{22} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$
$q_1 = \left[\begin{pmatrix} 1 & 2 \\ 1 & 1 \end{pmatrix} + \begin{pmatrix} 3 & 1 \\ 2 & 1 \end{pmatrix}\right] * \left[\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} + \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}\right]$
$= \begin{pmatrix} 4 & 3 \\ 3 & 2 \end{pmatrix} * \begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix}$ — (1)
A
B
Perform recursive call for (1) assuming
$a_{11} = 4$ $a_{12} = 3$ $a_{21} = 3$ $a_{22} = 2$
$b_{11} = 2$ $b_{12} = 2$ $b_{21} = 2$ $b_{22} = 2$

Example

$q_1 = (4+2) \times (2+2) = 6 \times 4 = 24$

$q_2 = (3+2) * 2 = 5 \times 2 = 10$

$q_3 = 4 * (2-2) = 0$

$q_4 = 2 * (2-2) = 0$

$q_5 = (4+3) * 2 = 14$

$q_6 = (3-4) * (2+2) = -4$

$q_7 = (3-2) * (2+2) = 4$

$c_{11} = 24 + 0 - 14 + 4 = 14$

$c_{12} = 0 + 14 = 14$

$c_{21} = 10 + 0 = 10$

$c_{22} = 24 + 0 - 10 + (-4) = 10$

Solution for (1) $\Rightarrow \begin{pmatrix} 14 & 14 \\ 10 & 10 \end{pmatrix}_{2\times2}$

$q_2 = \begin{pmatrix} 2 & 1 \\ 1 & 4 \end{pmatrix} + \begin{pmatrix} 3 & 1 \\ 2 & 1 \end{pmatrix} * \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$

$= \begin{pmatrix} 5 & 2 \\ 3 & 5 \end{pmatrix} * \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ — (2)

Example

Perform recursive call on (2) with

$a_{11}=5 \quad a_{12}=2 \quad a_{21}=3 \quad a_{22}=5$

$b_{11}=1 \quad b_{12}=1 \quad b_{21}=1 \quad b_{22}=1$

$q_1 = (5+5)*(1+1) = 20$

$c_2 = (3+5)*1 = 8$

$q_3 = 5+(1-1) = 0$

$q_4 = 5+(1-1) = 0$

$q_5 = (5+2)*1 = 7$

$q_6 = (3-5)*(1+1) = -4$

$q_7 = (2-5)*(1+1) = -6$

$c_{11} = 20+0-7-6 = 7$

$c_{12} = 0+7 = 7$

$c_{21} = 8+0 = 8$

$c_{22} = 20+0-8+4 = 8$

Solution for (2) $= \begin{pmatrix} 7 & 7 \\ 8 & 8 \end{pmatrix}$

$q_3 = \begin{pmatrix} 1 & 2 \\ 1 & 1 \end{pmatrix} * \left[\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} - \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \right]$

$= \begin{pmatrix} 1 & 2 \\ 1 & 1 \end{pmatrix} * \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$ ⇒ On recursive call it returning $\begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$

Example

$q_4 = \begin{pmatrix} 3 & 1 \\ 2 & 1 \end{pmatrix} * \left[\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} + \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \right]$

$= \begin{pmatrix} 3 & 1 \\ 2 & 1 \end{pmatrix} * \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$ → on recursive call it returns a 0.

$q_5 = \left[\begin{pmatrix} 1 & 2 \\ 1 & 1 \end{pmatrix} + \begin{pmatrix} 3 & 4 \\ 2 & 3 \end{pmatrix} \right] * \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$

$= \underset{A}{\begin{pmatrix} 4 & 6 \\ 3 & 4 \end{pmatrix}} * \underset{B}{\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}}$ — (3)

On performing recursive call for (3) with

$a_{11} = 4 \quad a_{12} = 6 \quad a_{21} = 3 \quad a_{22} = 4$

$b_{11} = 1 \quad b_{12} = 1 \quad b_{21} = 1 \quad b_{22} = 1$

$q_1 = (4 + 4) * (1 + 1) = 16$

$q_2 = (3 + 4) * 1 = 7$

$q_3 = 4 * (1 - 1) = 0$

$q_4 = 4 * (1 - 1) = 0$

$q_5 = (4 + 6) * 1 = 10$

$q_6 = (3 - 4) * (1 + 1) = -2$

$q_7 = (6 - 4) * (1 + 1) = 4$

Example

$c_{11} = 16 + 0 - 10 + 4 = 10$

$c_{12} = 0 + 0 = 10$

$c_{21} = 7 + 0 = 7$

$c_{22} = 16 + 0 - 7 + (-2) = 7$

(3) solution is $\begin{pmatrix} 10 & 10 \\ 7 & 7 \end{pmatrix}$

$q_6 = \left[\begin{pmatrix} 2 & 1 \\ 1 & 4 \end{pmatrix} - \begin{pmatrix} 1 & 2 \\ 1 & 1 \end{pmatrix}\right] * \left[\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} + \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}\right]$

$= \underset{A}{\begin{pmatrix} 1 & -1 \\ 0 & 3 \end{pmatrix}} * \underset{B}{\begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix}}$ — (4)

perform recursive call on (4) with

$a_{11} = 1 \quad a_{12} = -1 \quad a_{21} = 0 \quad a_{22} = 3$

$b_{11} = 2 \quad b_{12} = 2 \quad b_{21} = 2 \quad b_{22} = 2$

$q_1 = (1 + 3) * (2 + 2) = 16$

$q_2 = (0 + 3) * 2 = 6$

$q_3 = 1 * (2 - 2) = 0$

$q_4 = 3 * (2 - 2) = 0$

$q_5 = (1 + -1) * 2 = 0$

$q_6 = (0 - 1) * (2 + 2) = -4$

$q_7 = (-1 - 3) * (2 + 2) = -16$

Example

$c_{11} = 16 + 0 - 0 + -16 = 0$

$c_{12} = 0 + 0 = 0$

$c_{21} = 6 + 0 = 6$

$c_{22} = 16 + 0 - 6 + (-4) = 6$

Solution for ④ is $\begin{pmatrix} 0 & 0 \\ 6 & 6 \end{pmatrix}$

$$q_7 = \left[\begin{pmatrix} 3 & 4 \\ 2 & 3 \end{pmatrix} - \begin{pmatrix} 3 & 1 \\ 2 & 1 \end{pmatrix}\right] * \left[\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} + \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}\right]$$

$$= \underset{A}{\begin{pmatrix} 0 & 3 \\ 0 & 2 \end{pmatrix}} * \underset{B}{\begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix}} \quad \text{— ⑤}$$

Recursive call on ⑤ with

$a_{11} = 0 \quad a_{12} = 3 \quad a_{21} = 0 \quad a_{22} = 2$

$b_{11} = 2 \quad b_{12} = 2 \quad b_{21} = 2 \quad b_{22} = 2$

$q_1 = (0 + 2) * (2 + 2) = 8$

$q_2 = (0 + 2) * 2 = 4$

$q_3 = 0 + (2 - 2) = 0$

$q_4 = 2 * (2 - 2) = 0$

Example

$q_5 = (0+3) * 2 = 6$

$q_6 = (0-0) * (2+2) = 0$

$q_7 = (3-2) * (2+2) = 4$

~~solution for (5) is~~

$c_{11} = 8+0-6+4 = 6$

$c_{12} = 0+6 = 6$

$c_{21} = 4+0 = 4$

$c_{22} = 8+0-4+0 = 4$

solution for (5) is $\begin{pmatrix} 6 & 6 \\ 4 & 4 \end{pmatrix}$

Finally, compute $c_{11}, c_{12}, c_{21}, c_{22}$.

$$c_{11} = \begin{pmatrix} 14 & 14 \\ 10 & 10 \end{pmatrix} + \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix} - \begin{pmatrix} 10 & 10 \\ 7 & 7 \end{pmatrix} + \begin{pmatrix} 6 & 6 \\ 4 & 4 \end{pmatrix}$$

$$= \begin{pmatrix} 10 & 10 \\ 7 & 7 \end{pmatrix}$$

$$c_{12} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} 10 & 10 \\ 7 & 7 \end{pmatrix}$$

$$= \begin{pmatrix} 10 & 10 \\ 7 & 7 \end{pmatrix}$$

Example

$$C_{21} = \begin{pmatrix} 7 & 7 \\ 8 & 8 \end{pmatrix} + \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$$

$$= \begin{pmatrix} 7 & 7 \\ 8 & 8 \end{pmatrix}$$

$$C_{22} = \begin{pmatrix} 14 & 14 \\ 10 & 10 \end{pmatrix} + \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix} - \begin{pmatrix} 7 & 7 \\ 8 & 8 \end{pmatrix} + \begin{pmatrix} 0 & 0 \\ 6 & 6 \end{pmatrix}$$

$$= \begin{pmatrix} 7 & 7 \\ 8 & 8 \end{pmatrix}$$

Final result for the given matrix multiplication is

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} 10 & 10 & 10 & 10 \\ 7 & 7 & 7 & 7 \\ 7 & 7 & 7 & 7 \\ 8 & 8 & 8 & 8 \end{bmatrix}_{4\times4}$$

Strassen's Algorithm

	Mult	Add	Recurrence Relation	Runtime
Regular	8	4	$T(n) = 8T(n/2) + O(n^2)$	$O(n^3)$
Strassen	7	18	$T(n) = 7T(n/2) + O(n^2)$	$O(n^{\log_2 7}) = O(n^{2.81})$

Strassen’s Algorithm

- No idea how Strassen came up with these combinations.
  - He probably realized that he wanted to determine each element in the product using less than 8 multiplications.
    - From there, he probably just played around with it.

- If we let T(n) be the running time of Strassen's algorithm, then it satisfies the following recurrence relation:
  - $T(n) = 7T(n/2) + O(n^2)$
    - It's important to note that the hidden constant in the $O(n^2)$ term is larger than the corresponding constant for the standard divide and conquer algorithm for this problem.
  - However, for large matrices this algorithm yields an improvement over the standard one with respect to time.

Closest Pair Algorithm

- Minimum Distance between Two Points
- You are given an array **arr[]** of n distinct points in a 2D plane, where each point is represented by its (x, y) coordinates. Find the **minimum Euclidean distance** between **two distinct points**.
- **Note:** For two points $A(p_x,q_x)$ and $B(p_y,q_y)$ the distance Euclidean between them is:

$$\text{Distance} = \sqrt{\left(p_x - q_x\right)^2 + \left(p_y - q_y\right)^2}$$

- Naïve approach – $O(n^2)$ – check all possible pairs and store minimum
- Divide and Conquer – O(nlogn)

Closest Pair Algorithm

- Sort the points by x-coordinate.
- Recursively divide the points into left and right subarrays until each subarrays has one or two points:
  - If one point, return infinity.
  - If two points, return their distance.
- Find the minimum distances **dl** and **dr** in the left and right subarrays, and set **d** as the smaller value between **dl** and **dr**.
- Check points near the dividing line (strip of width **d**) – to find cross pairs
- Return smallest distance found

Closest Pair Algorithm

```
CLOSEST_PAIR(P, n)
        if n <= 3
                return minimum distance by brute force // O(1)

        mid = n / 2
        dl = CLOSEST_PAIR(PL, mid)
        dr = CLOSEST_PAIR(PR, n - mid)
        d = minimum(dl, dr)
        // O(n) to find points in strip within distance d
        Build strip[] containing points from Py
            whose |x - mid_x| < d
        ds = StripClosest(strip, d)
return minimum(d, ds)
```

```
StripClosest(strip, d)

Let min = d
For i = 1 to strip.length:
      For j = i+1 to strip.length
          while (strip[j].y - strip[i].y < min):
                compute distance
                update min if smaller

Return min
```